

## **BÁO CÁO SAU BUỔI THỰC HÀNH\_SESSION 18**

**Môn học:** Microservices System Design

**Tên buổi thực hành:** Ngân hàng số "RikkeiBank API"

**Thời gian thực hành:** 120p

**Họ và tên sinh viên:** Lê Nhật Linh

**Mã sinh viên:** PTIT-HN-121

**Lớp:** HN-K24-CNTT2

**Nhóm:** 2

### **I. NỘI DUNG ĐÃ THỰC HÀNH**

#### **1. Trình bày các giải pháp**

Dựa trên yêu cầu chuyển đổi hệ thống ngân hàng từ Monolithic sang Microservices (MSA) nhằm đảm bảo khả năng mở rộng độc lập và chịu lỗi cao, nhóm 2 đã triển khai các giải pháp kiến trúc sau:

- Kiến trúc tổng thể (Infrastructure Services):
  - Spring Cloud Config Server: Quản lý cấu hình tập trung từ Git/Local cho tất cả các Microservices.
  - Service Registry (Netflix Eureka): Đăng ký và phát hiện dịch vụ tự động.
  - API Gateway (Spring Cloud Gateway): Định tuyến duy nhất cho Client, kết hợp Spring Cloud LoadBalancer và tích hợp bộ lọc xác thực JWT tập trung.
- Phân chia Microservices (Database-per-service):
  - identity-service: Quản lý người dùng, phân quyền (ADMIN, TELLER, CUSTOMER), cấp phát/thu hồi JWT Token và quản lý phiên làm việc duy trì.
  - customer-service: Quản lý hồ sơ thông tin khách hàng, nhân viên giao dịch.

- account-service: Quản lý tài khoản ngân hàng, biến động số dư ghi nợ/ghi có.
- transaction-service: Lưu vết nhật ký giao dịch, lịch sử chuyển khoản.
- notification-service: Nhận sự kiện biến động qua Kafka và gửi thông báo.
- Xử lý giao dịch phân tán (Distributed Transaction - Saga Pattern):
  - Sử dụng mô hình Saga Orchestrator (hoặc Choreography via Kafka) cho nghiệp vụ Chuyển Khoản. Tách giao dịch thành các bước: Trừ tiền Tài khoản nguồn -> Cộng tiền Tài khoản đích -> Tạo bản ghi Giao dịch.
  - Compensating Transaction (Hành động bù): Nếu bước cộng tiền hoặc tạo bản ghi lỗi, hệ thống tự động phát hành sự kiện hoàn tiền (Rollback) lại cho tài khoản nguồn, đảm bảo tính nhất quán cuối cùng (Eventual Consistency).
- Giao tiếp giữa các Service:
  - Đồng bộ (Synchronous): Dùng OpenFeign tích hợp Spring Cloud LoadBalancer để các service truy vấn thông tin theo tên dịch vụ trên Eureka.
  - Bất đồng bộ (Asynchronous - Event Driven): Sử dụng Spring WebFlux + Apache Kafka để phát hành sự kiện (e.g., MoneyTransferredEvent) tới notification-service giúp giảm liên kết lỏng (Loose Coupling).
- Tối ưu hiệu năng & Kháng lỗi:
  - Distributed Caching: Sử dụng Spring Cache + Redis theo chiến lược Cache-Aside (@Cacheable, @CachePut, @CacheEvict) tại account-service và customer-service để giảm tải Database khi truy vấn số dư/danh mục.
  - Fault Tolerance: Tích hợp Resilience4j Circuit Breaker với 3 trạng thái (CLOSED, OPEN, HALF\_OPEN) và cơ chế Fallback Method ngăn chặn lỗi dây chuyền (Cascading Failure).

## 2. Liệt kê các câu hỏi từ nhóm khác và câu hỏi phản biện

Câu hỏi 1 (Từ Nhóm 1): Trong kiến trúc Microservices, nếu account-service bị ngắt kết nối giữa chừng khi đang trừ tiền tài khoản nguồn thì Saga Pattern đảm bảo số dư không bị lệch như thế nào?

- Trả lời của Nhóm: Khi account-service trừ tiền thành công, một sự kiện hoặc bước trong Orchestrator được ghi nhận. Nếu bước tiếp theo (cộng tiền ở tài khoản đích) thất bại hoặc timeout, Orchestrator sẽ gửi một lệnh Compensating Transaction quay lại account-service để cộng lại đúng số tiền đã trừ. Dữ liệu không bị khóa cứng như 2PC (Two-Phase Commit) nhưng đảm bảo nhất quán cuối cùng.

Câu hỏi 2 (Từ Nhóm 3): Yêu cầu ADMIN có thể ép buộc một tài khoản bất kỳ "đăng xuất ngay lập tức" được thực hiện như thế nào khi sử dụng Stateless JWT?

- Trả lời của Nhóm: Mặc dù JWT là Stateless, nhóm triển khai giải pháp Blacklist Token trên Redis Cache. Khi ADMIN thu hồi quyền/ép đăng xuất, jti (JWT ID) hoặc token đó sẽ được lưu vào Redis với thời gian hết hạn (TTL) bằng thời gian sống còn lại của Token. Tại API Gateway, mọi request đi qua đều kiểm tra Token có nằm trong Redis Blacklist hay không.

## 3. Thực hành triển khai code:

Khởi chạy Hạ tầng (Infrastructure):

- Khởi chạy Config Server (port 8888), Eureka Server (port 8761), Spring Cloud Gateway (port 8080).
- Đã đăng ký thành công tất cả 5 services (identity-service, customer-service, account-service, transaction-service, notification-service) lên Dashboard Eureka.

### Thực thi nghiệp vụ qua API Gateway:

- POST `http://localhost:8080/api/v1/auth/login`: Lấy JWT Token cho CUSTOMER / ADMIN / TELLER.

- POST `http://localhost:8080/api/v1/accounts/transfer`: Thực thi Chuyển khoản qua Saga Pattern.
- GET `http://localhost:8080/api/v1/transactions/daily`: Giao dịch viên (TELLER) xem danh sách giao dịch trong ngày.

### **Minh chứng kịch bản Rollback & Resilience4j:**

- Giả lập lỗi ở khoản tài khoản đích → Kiểm tra Log Saga gửi sự kiện đền bù (Compensating) → Số dư tài khoản nguồn tự động hoàn về ban đầu.
- Tắt account-service → Request từ Gateway kích hoạt Resilience4j Fallback trả về HTTP 503 với JSON Custom Message thay vì sập toàn bộ hệ thống.

## **II. CÔNG VIỆC CÁC EM ĐÃ LÀM**

### **1. Công việc cá nhân:**

Bắt ngoại lệ tập trung (Global Exception Handling) bằng `@ControllerAdvice` / `@ExceptionHandler` trả về chuẩn mã lỗi JSON thống nhất cho API

### **2. Công việc nhóm:**

Tổ chức meeting nhóm cùng phân tích, thảo luận, phản biện về những yêu cầu trong mini project.

## **III. KẾT QUẢ CÁC EM ĐẠT ĐƯỢC**

**Link mini-project:**

## **IV. KHÓ KHĂN VÀ VẤN ĐỀ CÁC EM GẶP PHẢI**

- Xung đột phiên bản Dependency: Xảy ra lỗi tương thích giữa Spring Boot 3.x, Spring Cloud phiên bản mới và các thư viện Kafka/Resilience4j.

- Quản lý Transactional trong Saga Pattern: Ban đầu gặp khó khăn khi xử lý trường hợp timeout mạng (Network Timeout) – service nguồn không rõ service đích đã nhận được lệnh cộng tiền chưa, dẫn đến rủi ro trừ tiền 2 lần (Duplicate Transfer).
- Cấu hình Redis Distributed Lock: Khi nhiều request chuyển khoản song song truy cập cùng 1 tài khoản, xảy ra hiện tượng Race Condition gây sai lệch số dư.

## V. KINH NGHIỆM RÚT RA

### 1. Kinh nghiệm rút ra từ bài thực hành:

- Tính Idempotency (Đồng đẳng) trong Event-driven: Mọi Consumer nhận tin nhắn từ Kafka (như notification-service hay Saga Consumer) bắt buộc phải triển khai cơ chế kiểm tra Idempotency Key (dùng Transaction ID/Message ID). Nếu Kafka gửi lặp lại message, hệ thống không được xử lý cộng/trừ tiền 2 lần.
- Khai thác Redis Lock: Đối với tài chính ngân hàng, việc @Cacheable là chưa đủ, phải dùng Redisson (Redis Distributed Lock) để khóa tài khoản trong vài mili-giây khi xử lý giao dịch biến động số dư.
- Tránh "Distributed Monolith": Không lạm dụng gọi RestTemplate/FeignClient đồng bộ liên tục giữa các service, ưu tiên dùng Event/Kafka cho các tác vụ không yêu cầu phản hồi tức thì (như gửi SMS/Email thông báo).

### 2. Lưu ý các em cần note lại cho những buổi thực hành sau:

- Chuẩn bị tốt tất cả các phần trong yêu cầu
- Đặt câu hỏi giúp mở rộng kiến thức và nâng cao khả năng phản biện

## VI. ĐỀ XUẤT / KIẾN NGHỊ

Không

## VII. KẾT LUẬN

- Áp dụng: Đã áp dụng thành công kiến trúc Microservices chuẩn doanh nghiệp cho dự án "RikkeiBank API", tích hợp đủ các thành phần: Eureka, Gateway, Config Server, Saga Pattern, Kafka, Redis, Resilience4j, JWT Security.
- Khả năng hiểu bài: Tốt (Nắm vững luồng đi của dữ liệu qua Gateway, cơ chế rollback của Saga và xử lý race-condition trong hệ thống phân tán).